JavaScript - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

JAVASCRIPT — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Written with Java comparisons throughout, since you already know Java deeply

---

TOPIC 1: ES6+ (MODERN JAVASCRIPT FEATURES)

---

1. Introduction

1.1 What is it?

"ES6" (ECMAScript 2015) was a MAJOR JavaScript language update — introducing features that modernized the language dramatically (similar in spirit to how Java 8's Lambdas/Streams modernized Core Java). "ES6+" refers to ES6 and every yearly release since (ES2016, ES2017, ... up through today).

1.2 Why it matters

Before ES6, JavaScript lacked many conveniences Java developers take for granted — proper block scoping, classes, destructuring, template strings. ES6+ closed this gap significantly, and virtually ALL modern JavaScript/React code (including everything in the React section) relies on these features.

2. `let`/`const` vs `var` — Block Scoping (Fixing a Real Historical Problem)

```
// OLD (var) - FUNCTION-scoped, NOT block-scoped - a classic source of bugs:
for (var i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 100);
}
// prints: 3, 3, 3 (ALL closures share the SAME 'i', which is 3 by the time they run!)

// MODERN (let) - BLOCK-scoped, exactly like Java's for-loop variable scoping:
for (let i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 100);
}
// prints: 0, 1, 2 (EACH iteration gets its OWN 'i', as most developers naturally expect)
```

Keyword	Scope	Reassignable?	Java Equivalent
`var` (legacy, avoid)	Function-scoped	Yes	Nothing quite like it — closer to a variable hoisted to the top
`let`	Block-scoped `{ }`	Yes	A regular local variable
`const`	Block-scoped `{ }`	No (cannot REASSIGN the binding — but object/array CONTENTS can still change)	A `final` local variable

3. Arrow Functions

```
// Traditional function
function add(a, b) { return a + b; }

// Arrow function (ES6) - more concise, and has DIFFERENT 'this' binding (covered in edge cases)
const add = (a, b) => a + b;

// Very similar in SPIRIT to a Java lambda expression:
list.forEach(item => console.log(item)); // JavaScript
list.forEach(item -> System.out.println(item)); // Java
```

4. Template Literals

```
const name = "Alice";
const salary = 75000;
console.log(`${name} earns $${salary.toLocaleString()} per year.`); // backticks + ${} - like Java's
text blocks + String.format combined
```

5. Destructuring (Recap from the React Section, Formalized)

```
const employee = { name: "Alice", department: "Engineering", salary: 75000 };
const { name, salary } = employee; // object destructuring

const numbers = [1, 2, 3];
const [first, second] = numbers; // array destructuring
```

6. Spread and Rest Operators (`...`)

```
const original = { name: "Alice", salary: 75000 };
const updated = { ...original, salary: 80000 }; // SPREAD - creates a NEW object, copying + overriding
fields
// (this is EXACTLY what you did in the React Forms topic!)

function sum(...numbers) { // REST - collects ALL arguments into an array
return numbers.reduce((total, n) => total + n, 0); // like Java's varargs (int... numbers)
}
sum(1, 2, 3); // 6
```

7. Real Life Analogy

- **Restaurant**: `const` is like a permanently RESERVED table number for the evening — you can't reassign WHICH table it refers to, but the ACTIVITY happening AT that table (the object's internal contents) can still change throughout the night. The spread operator is like quickly photocopying an existing order form and just CROSSING OUT/updating ONE line, instead of rewriting the entire form from scratch.

8. Edge Cases

Case	Result
Reassigning a `const` binding directly	`TypeError: Assignment to constant variable`
MUTATING an object/array declared with `const`	Perfectly LEGAL — `const` only locks the BINDING (the reference), not the object's internal contents, exactly like Java's `final` on an object reference
Using `var` inside a loop with an async callback	Classic bug (shown in Section 2) — all callbacks share the SAME variable, which has its FINAL value by the time they execute

9. Frequently Asked Interview Questions

1. **What's the difference between `let`, `const`, and `var`?** `var` is function-scoped (legacy, generally avoided); `let` and `const` are block-scoped; `const` additionally cannot be reassigned (though its contents, if an object/array, can still be mutated).
2. **Does `const` make an object fully immutable?** No — it only prevents REASSIGNING the variable itself; the object's own properties can still be changed, exactly like Java's `final` on a reference type.
3. **What does the spread operator (`...`) do when used on an object?** Copies all of that object's own properties into a NEW object, commonly combined with overriding specific fields (`{...original, field: newValue}`).

TOPIC 2: DOM (DOCUMENT OBJECT MODEL)

---

1. Introduction

What is it? The DOM is the BROWSER'S in-memory, tree-structured representation of an HTML page — every element (`<div>`, `<p>`, `<button>`) is a NODE in this tree, and JavaScript can read/modify it directly. React (previous section) works BY generating and diffing against this SAME underlying DOM, just automatically instead of manually.

2. Code Example — Direct DOM Manipulation (The "Before React" Approach)

```
const heading = document.getElementById("main-heading");
heading.textContent = "Hello, World!"; // change text content
heading.style.color = "blue"; // change styling directly
heading.classList.add("highlighted"); // add a CSS class

const button = document.querySelector(".submit-btn");
button.addEventListener("click", () => { // attach an event handler
console.log("Button clicked!");
});

const newParagraph = document.createElement("p"); // create a NEW element
newParagraph.textContent = "Dynamically added!";
document.body.appendChild(newParagraph); // insert it into the page
```

3. Why React Exists (Recap, Now Grounded in DOM Terms)

Manually keeping the DOM in sync with changing DATA (as shown above) becomes unmanageable at scale — React's **Virtual DOM** is an in-memory, lightweight COPY of the real DOM; when your data changes, React computes the MINIMAL set of actual DOM updates needed (a process called "diffing" / "reconciliation") and applies ONLY those changes, rather than you manually tracking every needed update yourself.

4. Real Life Analogy

- **Restaurant**: The DOM is like the ACTUAL physical layout of tables/decorations in a restaurant — directly rearranging chairs and dishes with your own hands (manual DOM manipulation) works for small changes, but becomes chaotic to coordinate perfectly as the restaurant (application) grows; React's Virtual DOM is like having an automated planning system that decides the MINIMAL, most efficient set of physical rearrangements needed, rather than you personally tracking every single change by hand.

5. Frequently Asked Interview Questions

1. **What is the DOM?** The browser's in-memory, tree-structured representation of an HTML page, which JavaScript can read and modify directly.
2. **What is the "Virtual DOM," and why does React use it?** A lightweight, in-memory copy of the real DOM that React uses to compute the MINIMAL set of actual DOM changes needed when data updates, avoiding the complexity/inefficiency of manual DOM manipulation at scale.

TOPIC 3: EVENTS

---

1. Introduction

What is it? Events are how JavaScript reacts to USER INTERACTIONS (clicks, key presses, form submissions) and BROWSER occurrences (page load, network responses) — you REGISTER a function (a "handler" or "listener") to run WHEN a specific event occurs.

2. Code Example

```
document.querySelector("#myButton").addEventListener("click", function(event) {
console.log("Clicked!", event.target); // 'event' carries details about what happened
});
```

3. Event Bubbling — A Frequently-Tested Concept

```
Clicking a <button> INSIDE a <div> INSIDE a <body>:

The click event FIRES on the button FIRST, then "BUBBLES UP" through
its ANCESTORS in order: button -> div -> body -> document

This lets you attach ONE listener on a PARENT element to catch events
from MANY child elements (called "event delegation") - efficient for
handling clicks on, say, hundreds of dynamically-added list items
without attaching hundreds of individual listeners.

document.querySelector("#employee-list").addEventListener("click", (event) => {
if (event.target.matches(".delete-btn")) { // check WHICH child was actually clicked
console.log("Delete clicked for:", event.target.dataset.employeeId);
}
});
```

4. `event.preventDefault()` — Recap from React's Forms Topic

```
form.addEventListener("submit", (event) => {
event.preventDefault(); // stops the BROWSER's default full-page-reload form submission
// ... handle the submission with JavaScript instead (e.g., a fetch call) ...
});
```

5. Real Life Analogy

- **Restaurant**: Event bubbling is like a customer's complaint about their DISH first being noticed by their immediate WAITER, then, if unresolved, escalating UP to the section manager, then the restaurant manager — a SINGLE manager stationed at a higher level can handle complaints bubbling up from MANY different tables, without needing a dedicated manager physically present at every single table.

6. Frequently Asked Interview Questions

1. **What is event bubbling?** An event fired on a specific element propagates UPWARD through its ancestor elements in the DOM tree, allowing a single listener on a parent to catch events originating from any of its descendants.
2. **What is "event delegation," and why is it useful?** Attaching ONE listener to a PARENT element (relying on bubbling) instead of separate listeners on many individual child elements — more efficient, and automatically works for children added DYNAMICALLY later.

3 **What does** `event.preventDefault()` **do?** Stops the browser's own default behavior for that event (like a form's full-page-reload submission), allowing custom JavaScript handling instead.

TOPIC 4: CLOSURES

---

1. Introduction

What is it? A closure is a function that "REMEMBERS" the variables from the SCOPE it was CREATED in, even after that outer scope has finished executing — one of JavaScript's most powerful (and most frequently interview-tested) features.

2. Code Example

```
function createCounter() {
let count = 0; // this variable is "enclosed" by the returned function

return function() {
count++; // the INNER function still has access to 'count',
return count; // even though createCounter() has ALREADY finished running!
};
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3 - 'count' PERSISTS between calls, private to THIS specific counter
instance
```

3. A Real, Practical Use Case — Private State (Encapsulation, JavaScript-Style)

```
function createBankAccount(initialBalance) {
let balance = initialBalance; // PRIVATE - not directly accessible from outside at all!

return {
deposit: (amount) => { balance += amount; },
withdraw: (amount) => { balance -= amount; },
getBalance: () => balance,
};
}

const account = createBankAccount(100);
account.deposit(50);
console.log(account.getBalance()); // 150
console.log(account.balance); // undefined - genuinely INACCESSIBLE from outside, real encapsulation!
```

> This achieves TRUE, real encapsulation (Core Java's Encapsulation topic) WITHOUT needing classes/`private` keywords at all — the closure itself is what hides `balance` from outside access.

4. Real Life Analogy

- **Bank**: A closure is like a personal SAFE DEPOSIT BOX with its OWN private combination that only ONE specific customer's transactions can access — even after the bank teller who originally SET UP that box has gone home for the day (the outer function has finished executing), the box (the closure) still remembers and protects its own private contents, accessible only through the specific procedures (returned functions) the bank provided for it.

5. Edge Cases

Case	Result
Multiple closures created from the SAME outer function call	Each gets its OWN independent copy of the enclosed variables (as shown — `counter` tracks its OWN separate `count`)
The classic `var` in a loop bug (Topic 1, Section 2)	A DIRECT consequence of closures + `var`'s function-scoping — every closure captures the SAME shared variable

6. Frequently Asked Interview Questions

1. **What is a closure?** A function that retains access to variables from the scope it was created in, even after that outer scope has finished executing.
2. **How can closures be used to achieve encapsulation in JavaScript?** By having an outer function declare "private" variables and return inner functions that reference them — those variables become genuinely inaccessible from outside code, since there's no direct reference to them at all.
3. **Why did the classic `var`-in-a-loop bug happen, in terms of closures?** All the closures created inside the loop captured the SAME shared `var` variable (function-scoped, not block-scoped), so by the time they actually ran, they all saw its FINAL value.

TOPIC 5: PROMISES

---

1. Introduction

What is it? A Promise represents the EVENTUAL result of an ASYNCHRONOUS operation (like a network request) — conceptually very similar to Java's `CompletableFuture` (Core Java's Multithreading section) — existing in one of THREE states: **pending**, **fulfilled** (succeeded), or **rejected** (failed).

2. Code Example

```
function fetchEmployee(id) {
return new Promise((resolve, reject) => {
fetch(`http://localhost:8080/api/employees/${id}`)
.then(response => {
if (response.ok) {
resolve(response.json()); // SUCCESS - "resolve" the promise with the result
} else {
reject(new Error("Employee not found")); // FAILURE - "reject" the promise with an error
}
});
});
}

fetchEmployee(1)
.then(employee => console.log("Got employee:", employee)) // runs on SUCCESS
.catch(error => console.error("Failed:", error.message)); // runs on FAILURE
```

3. Chaining Promises

```
fetch("http://localhost:8080/api/employees/1")
.then(response => response.json()) // each .then() returns a NEW promise, enabling chaining
.then(employee => fetch(`http://localhost:8080/api/departments/${employee.deptId}`))
.then(response => response.json())
.then(department => console.log("Department:", department.name))
.catch(error => console.error("Something failed:", error)); // ONE catch handles errors from ANY step
above
```

4. `Promise.all()` — Running Multiple Promises Concurrently

```
Promise.all([
fetch("http://localhost:8080/api/employees/1").then(r => r.json()),
fetch("http://localhost:8080/api/employees/2").then(r => r.json()),
]).then(([employee1, employee2]) => {
console.log(employee1, employee2); // waits for BOTH to complete, exactly like Java's
CompletableFuture.allOf()
});
```

5. Promises vs Java's `CompletableFuture` — Comparison Table

Aspect	JavaScript Promise	Java `CompletableFuture`
Represents	The eventual result of an async operation	Same concept
Chaining	`.then()` / `.catch()`	`.thenApply()` / `.exceptionally()`
Running multiple concurrently	`Promise.all([...])`	`CompletableFuture.allOf(...)`
Underlying concurrency model	Single-threaded event loop (Topic 8)	Real OS threads (thread pool)

6. Real Life Analogy

- **Restaurant**: A Promise is like a numbered PICKUP TICKET given for a food order that's still being prepared — you don't get the food immediately, but you HOLD a ticket representing "this will EVENTUALLY either be ready (fulfilled) or the kitchen will run out of that ingredient and cancel it (rejected)" — and you can specify exactly what to do in EITHER case ahead of time.

7. Frequently Asked Interview Questions

1. **What is a Promise?** An object representing the eventual result of an asynchronous operation, existing in one of three states: pending, fulfilled, or rejected.
2. **What's the JavaScript equivalent of Java's `CompletableFuture.allOf()`?** `Promise.all([...])` — waits for multiple promises to all complete.
3. **What does `.catch()` at the END of a `.then()` chain do?** Catches an error from ANY of the preceding steps in the chain, not just the immediately preceding one.

TOPIC 6: ASYNC/AWAIT

---

1. Introduction

What is it? `async`/`await` is SYNTACTIC SUGAR over Promises, letting you write asynchronous code that READS like straightforward, sequential (synchronous) code — dramatically more readable than long `.then()` chains, without actually changing the underlying async behavior at all.

2. Code Example — Before and After

```
// WITH .then() chaining (Topic 5):
function loadEmployeeAndDept(id) {
return fetch(`http://localhost:8080/api/employees/${id}`)
.then(response => response.json())
.then(employee => fetch(`http://localhost:8080/api/departments/${employee.deptId}`))
.then(response => response.json())
.then(department => console.log("Department:", department.name))
.catch(error => console.error("Failed:", error));
}

// WITH async/await - reads almost like SYNCHRONOUS Java code!
async function loadEmployeeAndDept(id) {
try {
const empResponse = await fetch(`http://localhost:8080/api/employees/${id}`);
const employee = await empResponse.json();

const deptResponse = await fetch(`http://localhost:8080/api/departments/${employee.deptId}`);
const department = await deptResponse.json();

console.log("Department:", department.name);
} catch (error) {
console.error("Failed:", error);
}
}
```

3. Key Rules

- `await` can ONLY be used INSIDE a function marked `async`.

- An `async` function ALWAYS returns a Promise (even if you `return` a plain value directly — it's automatically wrapped).
- `try/catch` works NATURALLY with `await` — exactly like Java's exception handling, unlike `.then()`/`.catch()`'s more indirect chaining style.

4. Real Life Analogy

- **Restaurant**: `.then()` chaining is like a waiter juggling MULTIPLE separate pickup tickets, remembering to check back on each one at the right moment. `async`/`await` is like the SAME waiter simply STANDING and WAITING at the counter for each dish in turn, in a natural, straightforward, top-to-bottom sequence — easier to read and reason about, even though the KITCHEN is still working asynchronously behind the scenes either way.

5. Common Mistakes

- Forgetting `await` on an async call, resulting in working with the Promise OBJECT itself instead of its resolved VALUE.
- Forgetting `try/catch` around `await` calls, letting a rejected promise become an unhandled error.

6. Frequently Asked Interview Questions

1 **What is `async`/`await`?** Syntactic sugar over Promises, letting asynchronous code be written and read in a sequential, synchronous-looking style.

2 **What does an `async` function always return?** A Promise — even a plain returned value is automatically wrapped in one.

3 **How do you handle errors in `async`/`await` code?** With a regular `try/catch` block, exactly like synchronous Java exception handling.

4 **Where can `await` be used?** Only inside a function declared with the `async` keyword.

TOPIC 7: MODULES

---

1. Introduction

What is it? JavaScript Modules let you SPLIT code across MULTIPLE files, EXPORTING specific functions/values from one file and IMPORTING them in another — directly analogous to Java's package/import system (Core Java's Packages topic), solving the SAME "organize large codebases, avoid naming collisions" problem.

2. Code Example

```
// employeeService.js - EXPORTING
export function getEmployee(id) {
return fetch(`http://localhost:8080/api/employees/${id}`).then(r => r.json());
}

export const API_BASE_URL = "http://localhost:8080/api";

export default function EmployeeCard({ name }) { // a DEFAULT export - only ONE per file
return <div>{name}</div>;
}

// EmployeeList.js - IMPORTING
import EmployeeCard, { getEmployee, API_BASE_URL } from "./employeeService.js";

getEmployee(1).then(employee => console.log(employee));
```

3. Named Export vs Default Export — Comparison

Aspect	Named Export (`export function ...`)	Default Export (`export default ...`)
How many per file	Multiple allowed	Only ONE per file
Import syntax	`import { name } from "..."` (name must MATCH)	`import anyNameYouWant from "..."` (name can be ANYTHING)
Java parallel	Like importing SPECIFIC classes from a package	Like a package having ONE "primary" public class

4. Real Life Analogy

- **Library**: Modules are like organizing a large book collection into separate, clearly-labeled SECTIONS (files) — each section EXPORTS (makes available) specific books, and other sections IMPORT (borrow) exactly the specific books they need, rather than everything being dumped into ONE giant, disorganized pile.

5. Frequently Asked Interview Questions

1. **What problem do JavaScript Modules solve?** Splitting code across multiple files with explicit exports/imports, avoiding naming collisions and disorganized global scope — the same problem Java's package system solves.
2. **What's the difference between a named export and a default export?** A file can have MULTIPLE named exports (imported by their exact matching name) but only ONE default export (imported under any name the importer chooses).

TOPIC 8: FETCH API

---

1. Introduction

What is it? The Fetch API is the browser's BUILT-IN mechanism for making HTTP requests (calling REST APIs, like the Spring Boot backend built throughout this course) — the modern replacement for the older `XMLHttpRequest`, and the SAME `fetch()` function used throughout the React section.

2. Code Example — Full CRUD via Fetch

```
// GET
const employees = await fetch("http://localhost:8080/api/employees").then(r => r.json());

// POST
const newEmployee = await fetch("http://localhost:8080/api/employees", {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ firstName: "Alice", salary: 75000 }),
}).then(r => r.json());

// PUT with an Authorization header (recall Spring Security's JWT topic!)
await fetch("http://localhost:8080/api/employees/1", {
method: "PUT",
headers: {
"Content-Type": "application/json",
"Authorization": `Bearer ${jwtToken}`,
},
body: JSON.stringify({ firstName: "Alice", salary: 80000 }),
});

// DELETE
await fetch("http://localhost:8080/api/employees/1", { method: "DELETE" });
```

3. A Critical Gotcha — `fetch()` Does NOT Reject on HTTP Error Status Codes

```
const response = await fetch("http://localhost:8080/api/employees/9999");
// even for a 404 response, fetch() does NOT throw/reject - you must check response.ok MANUALLY!
if (!response.ok) {
throw new Error(`Request failed: ${response.status}`);
}
```

> This is a FREQUENTLY-missed gotcha: `fetch()` only rejects for genuine NETWORK failures (no connection at all) — a `404`/`500` response is still considered a "successful" fetch AS FAR AS the Promise itself is concerned; you must explicitly check `response.ok` (or `response.status`) yourself.

4. Real Life Analogy

- **Restaurant**: `fetch()` is like sending a food delivery request — the DELIVERY ITSELF succeeding (the Promise resolving) just means the delivery person REACHED the destination and handed something over; it says NOTHING about whether the KITCHEN actually had your item in stock (a 404) or made an error (a 500) — you must open the package and CHECK its actual contents/status yourself.

5. Frequently Asked Interview Questions

1. **What is the Fetch API?** The browser's built-in mechanism for making HTTP requests, the modern replacement for `XMLHttpRequest`.
2. **Does `fetch()` throw/reject for HTTP error responses like 404 or 500?** No — it only rejects for genuine network failures; HTTP error statuses must be checked manually via `response.ok`/`response.status`.
3. **How would you send an Authorization header with a JWT via fetch?** Include it in the `headers` object: `"Authorization": \Bearer ${token}``.

TOPIC 9: EVENT LOOP

---

1. Introduction

What is it? The Event Loop is the mechanism that lets JavaScript — a SINGLE-THREADED language — handle asynchronous operations (network requests, timers) WITHOUT blocking, by continuously checking whether the CALL STACK is empty and, if so, pulling the next queued CALLBACK to execute.

2. The Critical Contrast With Java — A Frequently-Tested Concept

```
JAVA: multi-threaded by design (Core Java's Multithreading section) - can run
MULTIPLE pieces of code TRULY simultaneously across real OS threads.

JAVASCRIPT (in the browser): SINGLE-THREADED - only ONE piece of JavaScript
code runs at any given instant. Asynchronous operations (fetch, setTimeout)
don't run "in parallel" WITHIN JavaScript itself - they're handled by the
BROWSER (or Node.js) behind the scenes, with their CALLBACK functions
queued up to run LATER, once the current synchronous code finishes.
```

3. Visualizing the Event Loop

```
CALL STACK (synchronous code executes here, one thing at a time)
|
v
console.log("1");
setTimeout(() => console.log("2"), 0); // callback is handed OFF to the browser, NOT run immediately
console.log("3");

OUTPUT: 1, 3, 2 (NOT 1, 2, 3!)
- "1" and "3" run IMMEDIATELY (synchronous, on the call stack)
- the setTimeout callback is QUEUED, and only runs AFTER the call stack
is COMPLETELY EMPTY (i.e., after "3" has already printed), even with
a delay of 0ms!
```

4. Microtasks vs Macrotasks (A Deeper, Interview-Favorite Detail)

```
console.log("1");
setTimeout(() => console.log("2"), 0); // a MACROTASK
Promise.resolve().then(() => console.log("3")); // a MICROTASK
console.log("4");

OUTPUT: 1, 4, 3, 2
- Microtasks (Promise callbacks) are ALWAYS processed BEFORE the next
macrotask (setTimeout callbacks), even if the macrotask was scheduled FIRST!
```

5. Real Life Analogy

- **Restaurant**: The Event Loop is like a SINGLE waiter (one thread) who can only do ONE thing at a time — take an order, deliver food, handle a complaint — but who NEVER just stands idle waiting for the kitchen; instead, they hand off a slow task to the kitchen (an async operation) and IMMEDIATELY move on to serve OTHER tables, only coming BACK to that original task once the kitchen signals it's ready AND the waiter has finished whatever they were currently doing.

6. Edge Cases

Case	Result
`setTimeout(fn, 0)`	Does NOT run immediately — still queued as a macrotask, only running after the current synchronous code AND all pending microtasks finish
A long-running, purely SYNCHRONOUS loop	BLOCKS the entire single thread — the browser UI freezes, and NO async callbacks (even ones "ready" to run) can execute until that loop finishes

7. Common Mistakes

- Assuming `setTimeout(fn, 0)` runs "immediately" — it still waits for the call stack to clear and any pending microtasks to finish first.
- Writing long, blocking synchronous JavaScript loops, freezing the entire single-threaded UI (no separate "background thread" rescues you, unlike Java's multithreading).

8. Frequently Asked Interview Questions

1. **Is JavaScript single-threaded or multi-threaded (in the browser)?** Single-threaded — only one piece of JavaScript code executes at any given instant.
2. **How does JavaScript handle asynchronous operations if it's single-threaded?** The Event Loop lets the browser/runtime handle async operations (network calls, timers) OUTSIDE of JavaScript's own execution, queuing their callback functions to run once the call stack is empty.
3. **What's the difference between a microtask and a macrotask?** Microtasks (like Promise callbacks) are ALWAYS fully processed before the NEXT macrotask (like a `setTimeout` callback) runs, regardless of which was scheduled first.

4 **Why does a long, blocking synchronous loop freeze a web page's UI?** Because JavaScript is single-threaded — that one thread is completely occupied by the loop, leaving no capacity to process ANY other queued callbacks (including UI updates) until the loop finishes.

9. Revision Notes (Entire JavaScript Section Consolidated)

```
JAVASCRIPT — CHEAT SHEET
============================
ES6+: let/const (block-scoped, replacing var's function-scoping bugs),
arrow functions (like Java lambdas), template literals (`${}`),
destructuring, spread/rest (...)
const locks the BINDING, not the object's contents (like Java's final)

DOM: browser's in-memory HTML tree; React's Virtual DOM automates keeping
it in sync with data, instead of manual document.querySelector() updates

EVENTS: addEventListener, event BUBBLING (child -> parent), event delegation
(one parent listener catches many children's events), preventDefault()

CLOSURES: a function "remembers" its creation scope's variables even after
that scope finishes - enables PRIVATE state/encapsulation without classes

PROMISES: pending/fulfilled/rejected, .then()/.catch() chaining,
Promise.all() (like Java's CompletableFuture.allOf())
ASYNC/AWAIT: syntactic sugar over Promises - reads like sequential code,
try/catch works naturally, await ONLY inside async functions

MODULES: export/import - same "organize code, avoid collisions" goal as
Java packages; named exports (multiple, exact-name import) vs default
export (ONE per file, import under any name)

FETCH API: the browser's HTTP client
GOTCHA: fetch() does NOT reject on 404/500 - only on genuine network
failure - ALWAYS check response.ok manually

EVENT LOOP: JavaScript is SINGLE-THREADED (unlike Java) - async callbacks
queue and run only once the call stack is empty
MICROTASKS (Promises) always run before the NEXT MACROTASK (setTimeout),
even setTimeout(fn, 0) doesn't run "immediately"
a blocking sync loop freezes EVERYTHING (no other thread to save you)
```

10. Homework (Covers Entire JavaScript Section)

1 Reproduce the classic `var`-in-a-loop closure bug, then fix it using `let`.

2 Write a `createBankAccount` closure-based "class," proving `balance` is genuinely inaccessible from outside.

3 Rewrite a `.then()`-chained sequence of 3 dependent `fetch()` calls (to your Spring Boot API) using `async`/`await` with proper `try/catch` error handling.

4 Predict, then verify, the output order of a code snippet mixing `console.log`, `setTimeout`, and `Promise.resolve().then()`.

FINAL SUMMARY — JAVASCRIPT COMPLETE

This concludes the **JavaScript** section — 9 topics, throughout drawing direct comparisons to concepts you already know from Java:

1. **ES6+** — `let`/`const`, arrow functions, destructuring, spread/rest.
2. **DOM** — the browser's page representation React automates updates to.
3. **Events** — bubbling, delegation, `preventDefault()`.
4. **Closures** — private state/encapsulation without classes.
5. **Promises** — the `CompletableFuture` parallel, chaining, `Promise.all()`.
6. **Async/Await** — synchronous-reading syntax over Promises.
7. **Modules** — the `export`/`import` parallel to Java packages.
8. **Fetch API** — the critical "doesn't reject on 404/500" gotcha.
9. **Event Loop** — JavaScript's single-threaded model, microtasks vs macrotasks.

You now have the language fundamentals underlying every line of React code from the previous section. Next: TypeScript, adding the compile-time type safety to JavaScript that Java has always had natively.

(END OF JAVASCRIPT MASTER NOTES — ready to proceed to TypeScript whenever you say "Next Topic.")